Tema 08

E/S en Archivos y Dispositivos

Programación en Lenguaje C

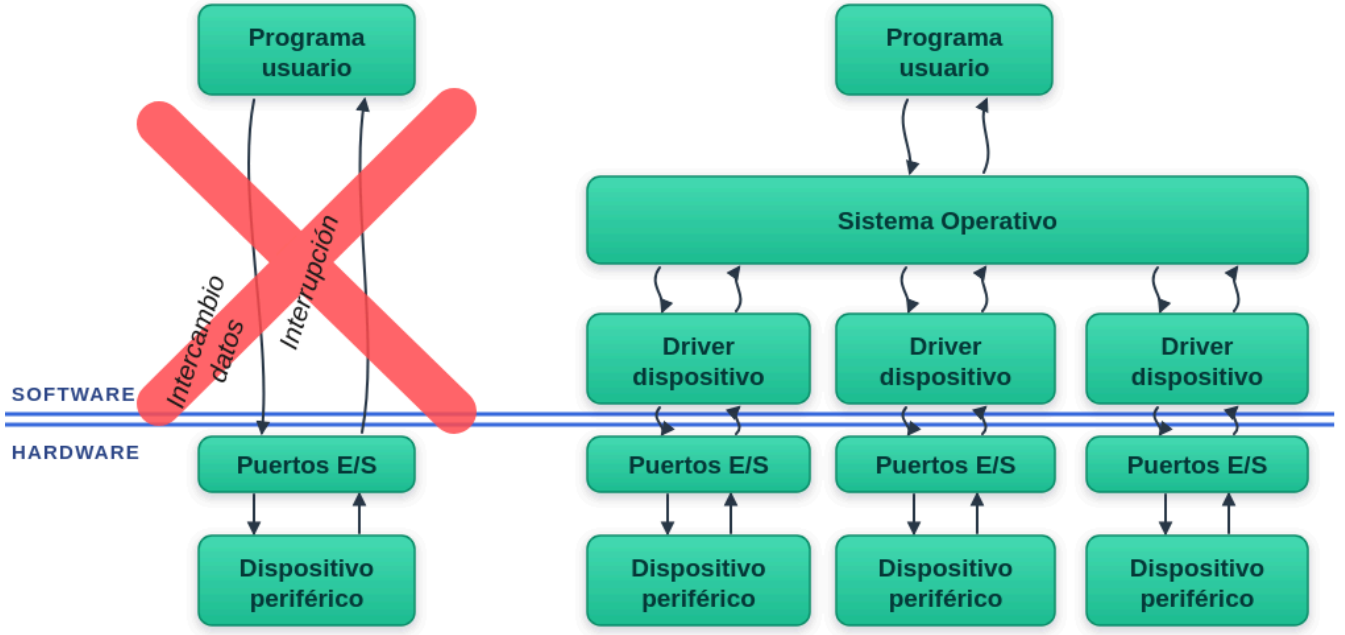
Ingeniería de Sistemas y Automática - Universidad de Oviedo

Índice del Tema

1. Concepto de Entrada/Salida por *Streams* y arquitectura SO/Driver
2. Tipos de Streams y denominación de dispositivos
3. Apertura y cierre de streams (`fopen`, `fclose`, tipo `FILE*`)
4. Entrada/Salida orientada a texto (`fgetc`, `fgets`, `fprintf`, `fscanf`)
5. Streams estándar de consola (`stdin`, `stdout`, `stderr`) y redirección
6. Entrada/Salida binaria (`fread`, `fwrite`)
7. Posicionamiento y control de errores (`fseek`, `ftell`, `feof`, `fflush`)
8. E/S de bajo nivel sin buffer (POSIX / Unix)

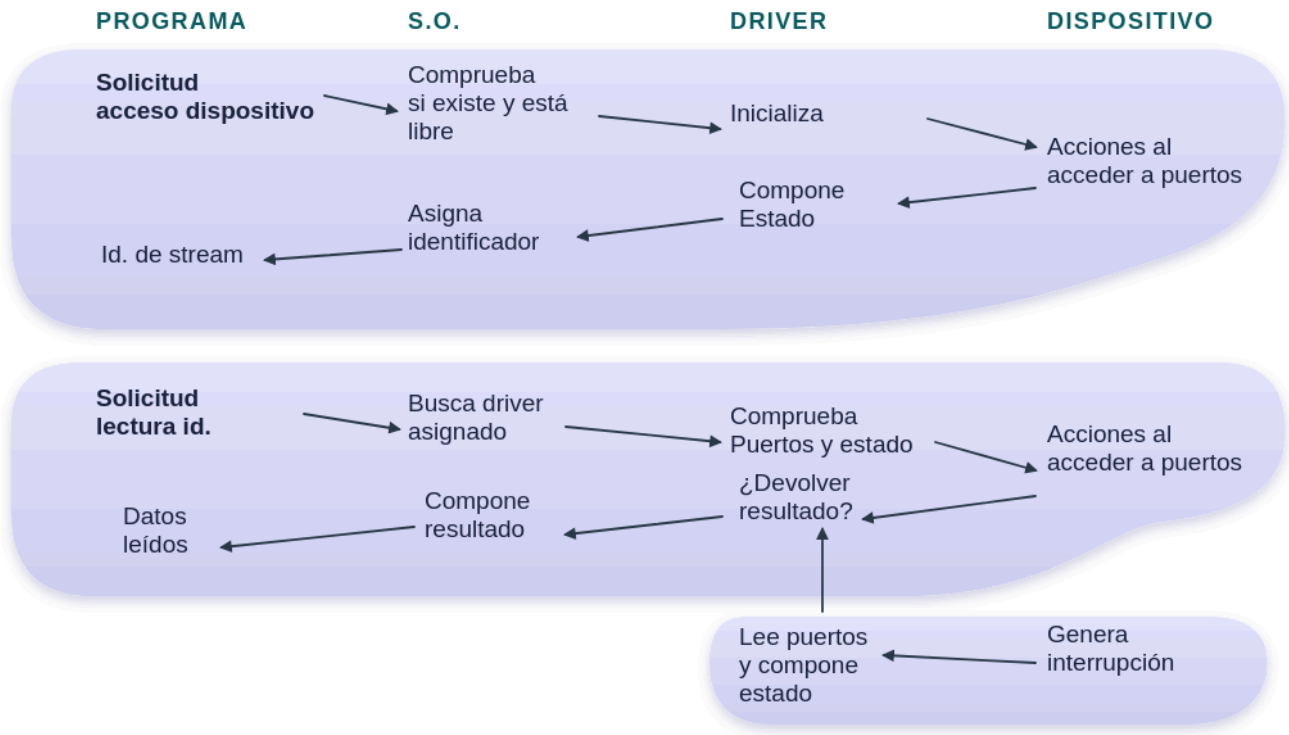
E/S por Stream

Permite a los programas intercambiar información con dispositivos periféricos mediante streams:



E/S por Stream

Funcionamiento de la E/S por stream:



E/S por Stream

Un **Stream** es una secuencia ordenada de bytes enviada a o recibida desde un dispositivo. Todos los dispositivos se tratan de forma homogénea.

Tipos de Streams:

- **Según dirección:**
  - Entrada
  - Salida
  - Entrada/Salida (bidireccional)
- **Según soporte:**
  - **Con memoria:** Archivos en disco
  - **Sin memoria:** Teclado, pantalla, puerto serie...
- **Según formato:**
  - Orientados a texto
  - Binarios

Identificación: Mediante cadena de caracteres (ej. `"COM1"`, `"/dev/ttyS0"`, `"C:\\datos.txt"`).

- El**Sistema Operativo** se encarga de la organización lógica (nombres, drivers, permisos, estado).
- Los **Drivers** se encargan del acceso físico (puertos, gestión de interrupciones).

Funciones para manejo de streams

- Permiten asignar un identificador a un stream, leer/escribir sobre él y liberar el stream asociado.
- Requieren incluir la librería `<stdio.h>`.
- El identificador de stream se declara como una variable de tipo `FILE*`.

- **Funciones de apertura / cierre de stream:**

```
FILE* fopen(const char* name, const char* mode);
```

  - **name:** Nombre del dispositivo. Si no existe, se supone que es un archivo en un dispositivo de almacenamiento.
  - **mode:** Modo de apertura: lectura / escritura / añadir, texto / binario.

```
int fclose(FILE* fid);
```

Funciones para manejo de streams

- Funciones de E/S en streams de texto:

  - E/S de caracteres:

```
int fgetc(FILE* fid);                          // Lee 1 carácter
int fputc(int car, FILE* fid);                 // Escribe 1 carácter
```

  - E/S de cadenas:

```
char* fgets(char* str, int n, FILE* fid);      // Lee 1 línea
int fputs(const char* str, FILE* fid);         // Escribe 1 línea
```

  - E/S con formato:

```
int fscanf(FILE* fid, char* fmt, ...);         // Lee con formato
int fprintf(FILE* fid, char* fmt, ...);// Escribe con formato
```

Streams por defecto para consola

- **Streams por defecto: permiten realizar E/S a consola con `printf()`, `scanf()`, `gets()`, `puts()`, `getchar()`, `putchar()`:**
  - **`stdin:`** entrada estándar
  - **`stdout:`** salida estándar
  - **`stderr:`** salida de error

- **`Ej: printf(...) ≡ fprintf(stdout, ...)`**

- **Estos streams están conectados por defecto a la consola, en modo texto (stdin a lectura, stdout y stderr a escritura), pero pueden redirigirse a otros dispositivos (ej. a archivo):**

```
Miprograma.exe < entrada.txt > salida.txt
```

Funciones para manejo de streams

- **Funciones de E/S en streams binarios:**

  - **Lectura de datos:**

```
int fread(void* buffer, size_t size, size_t count, FILE
*fid);
```

  - Lee un conjunto de count*size bytes del stream y los almacena en el mismo formato en el buffer

  - **Escritura de datos:**

```
int fwrite(const void* buffer, size_t size, size_t count,
FILE *fid);
```

  - Envía al stream un conjunto de count*size bytes indicados a partir de la dirección de memoria buffer

  - **En ambos casos:**

Funciones para manejo de streams

- **Funciones auxiliares para streams con almacenamiento (archivos):**

```
int feof(FILE* fid);
```

▶ Indica si se ha alcanzado el fin de archivo

```
void rewind(FILE* fid);
```

▶ Vuelve al principio del archivo

```
long ftell(FILE *fid);
```

▶ Devuelve la posición actual (en bytes) desde el principio del stream

```
int fseek(FILE *fid, long offset, int origin);
```

▶ Coloca el puntero en la posición (bytes) deseada a partir de:

- el principio: **origin = SEEK_SET**
- la posición actual: **origin = SEEK_CUR**
- el final: **origin = SEEK_END**

Funciones para manejo de streams

- **Funciones auxiliares para streams:**

```
int ferror(FILE* fid);
```

▶ Devuelve el código del último error (0 = ok)

```
void clearerr(FILE* fid);
```

▶ Borra el código del último error

```
int fflush(FILE *fid);
```

▶ Vacía los búferes intermedios con los que el S.O. gestiona la E/S física del dispositivo asociado al stream:

- Si estaba abierto para salida, asegura que dicha salida se escribe físicamente.
- Si estaba abierto para entrada, elimina los datos de los búferes intermedios.

Funciones de E/S sin buffer

- **Acceso directo al driver:**
  - Se realiza la E/S directamente al driver, sin conversiones ni búferes intermedios.
  - Identificador de dispositivo: tipo de datos **`int`**.
  - Dispositivos por defecto: **`stdin ≡ 0`**, **`stdout ≡ 1`**, **`stderr ≡ 2`**.

- **Funciones básicas de bajo nivel:**

```
open(), creat(), close(), read(), write(), tell(), ...
```

  - No forman parte del estándar ANSI, aunque están presentes en la mayoría de implementaciones.

- **Funciones especializadas en entornos estilo Unix:**

```
ioctl(), fcntl(), select()
```

  - Permiten control avanzado de dispositivos y operaciones de E/S no bloqueantes o multiplexadas.

Resumen del Tema

1. El modelo de Streams en C abstrae el intercambio de información con cualquier periférico o archivo.
2. `fopen` permite seleccionar modos de apertura en texto o binario.
3. Para ficheros formateados usar `fprintf`/`fscanf`; para grandes volúmenes de datos crudos usar `fwrite`/`fread`.
4. Comprobar siempre que `fopen` no devuelva `NULL` antes de operar sobre un stream.
5. Utilizar `fflush` cuando sea crítico asegurar la escritura inmediata sobre hardware.